

Intelligent Game-AI-Based Student Attendance Risk Prediction System

Techniques: Minimax (best/worst case) + Expectimax (probabilistic) with pruning & memoization

```
import math
import sys
from dataclasses import dataclass
from functools import lru_cache

THRESHOLD = 0.75          # minimum required attendance (75%)
LOW_LIMIT = 0.85          # P(Safe) >= 0.85 -> Low risk
MED_LIMIT = 0.50          # P(Safe) >= 0.50 -> Medium risk

@dataclass
class Student:
    name: str
    attended: int          # classes attended so far
    held: int              # classes held so far
    remaining: int         # classes remaining in semester
    p_attend: float        # probability of attending each future class

    @property
    def total(self):
        return self.held + self.remaining

# ----- Basic calculations -----
def current_percentage(s):
    return s.attended / s.held

def classes_needed(s):
    """Minimum future classes to attend to reach the threshold."""
    return max(0, math.ceil(THRESHOLD * s.total - s.attended))

def safe_bunks(s):
    """Maximum future classes the student can skip and still reach the threshold."""
    need = classes_needed(s)
    return max(0, s.remaining - need) if need <= s.remaining else 0

# ----- Minimax bounds -----
def best_case(s):
    """MAX player: student attends all remaining classes."""
    return (s.attended + s.remaining) / s.total

def worst_case(s):
    """MIN player: student is absent in all remaining classes."""
    return s.attended / s.total

# ----- Expectimax -----
def expectimax_success(s):
    """Probability of finishing with attendance >= threshold."""
    total = s.total
    p = s.p_attend
```

```

@lru_cache(maxsize=None)
def E(att, left):
    if left == 0:                                     # terminal node
        return 1.0 if att / total >= THRESHOLD else 0.0
    if (att + left) / total < THRESHOLD:               # prune: impossible
        return 0.0
    if att / total >= THRESHOLD:                       # prune: already
safe
        return 1.0
    return p * E(att + 1, left - 1) + (1 - p) * E(att, left - 1)

return E(s.attended, s.remaining)

# ----- Classification & advice -----
def classify(prob, best):
    if best < THRESHOLD:
        return "CRITICAL"
    if prob >= LOW_LIMIT:
        return "LOW RISK"
    if prob >= MED_LIMIT:
        return "MEDIUM RISK"
    return "HIGH RISK"

def advice(category, s):
    need = classes_needed(s)
    if category == "CRITICAL":
        return "Cannot reach 75% even with full attendance. Apply for
condonation / meet mentor."
    if category == "HIGH RISK":
        return f"Attend at least {need} of {s.remaining} remaining classes.
Counselling recommended."
    if category == "MEDIUM RISK":
        return f"Attend {need} of {s.remaining} remaining classes; avoid
more than {safe_bunks(s)} absences."
    return f"Safe. Can skip up to {safe_bunks(s)} classes, but regular
attendance is advised."

# ----- Input / Output -----
def sample_students():
    return [
        Student("Arun", 62, 70, 30, 0.90),
        Student("Bhavana", 48, 65, 35, 0.80),
        Student("Charan", 40, 66, 34, 0.75),
        Student("Divya", 55, 60, 40, 0.95),
        Student("Elango", 30, 62, 38, 0.70),
        Student("Farah", 45, 55, 45, 0.60),
    ]

def read_students():
    n = int(input("Number of students: "))
    data = []
    for i in range(n):
        print(f"\nStudent {i + 1}")
        name = input("  Name: ")
        att = int(input("  Classes attended so far: "))
        held = int(input("  Classes held so far: "))
        rem = int(input("  Classes remaining: "))

```

```

        p = float(input(" Probability of attending future classes (0-1):
"))
        if att > held or not 0 <= p <= 1:
            print(" Invalid data, skipping.")
            continue
        data.append(Student(name, att, held, rem, p))
    return data

def analyse(students):
    results = []
    for s in students:
        prob = expectimax_success(s)
        best = best_case(s)
        cat = classify(prob, best)
        results.append((s, prob, best, cat))
    return results

def print_table(results):
    print("\n" + "=" * 92)
    print(" ATTENDANCE RISK PREDICTION REPORT (Threshold = 75%)")
    print("=" * 92)
    print(f"{'Student':<9}{'Curr%':>7}{'Need':>8}{'Best%':>8}{'Worst%':>8}"
          f"{'P(Safe)':>9}{'Risk':>8} Category")
    print("-" * 92)
    for s, prob, best, cat in results:
        need = f"{classes_needed(s)}/{s.remaining}"
        print(f"{s.name:<9}{100 * current_percentage(s):7.2f}{need:>8}"
              f"{100 * best:8.2f}{100 * worst_case(s):8.2f}"
              f"{prob:9.4f}{100 * (1 - prob):8.2f} {cat}")

def print_advice(results):
    print("\n" + "=" * 92)
    print(" RECOMMENDATIONS")
    print("=" * 92)
    for s, prob, best, cat in results:
        print(f"{s.name:<9}[{cat}] {advice(cat, s)}")

def print_summary(results):
    counts = {"LOW RISK": 0, "MEDIUM RISK": 0, "HIGH RISK": 0, "CRITICAL":
0}
    for _, cat in results:
        counts[cat] += 1
    print("\n" + "=" * 92)
    print(" SUMMARY")
    print("=" * 92)
    for k, v in counts.items():
        print(f"{k:<12}: {v}")
    worst = max(results, key=lambda r: 1 - r[1])
    print(f"Highest risk student: {worst[0].name} ({100 * (1 -
worst[1]):.2f}%)")

def main():
    students = read_students() if "--input" in sys.argv else
sample_students()
    results = analyse(students)
    print_table(results)
    print_advice(results)
    print_summary(results)

```

```
if __name__ == "__main__":
    main()
```

Output

```
=====
=====
ATTENDANCE RISK PREDICTION REPORT (Threshold = 75%)
=====
=====
Student      Curr%      Need      Best%      Worst%      P(Safe)      Risk%      Category
-----
Arun          88.57      13/30      92.00      62.00      1.0000      0.00      LOW RISK
Bhavana       73.85      27/35      83.00      48.00      0.7450      25.50      MEDIUM RISK
Charan        60.61      35/34      74.00      40.00      0.0000      100.00      CRITICAL
Divya         91.67      20/40      95.00      55.00      1.0000      0.00      LOW RISK
Elango        48.39      45/38      68.00      30.00      0.0000      100.00      CRITICAL
Farah         81.82      30/45      90.00      45.00      0.2249      77.51      HIGH RISK

=====
=====
RECOMMENDATIONS
=====
=====
Arun          [LOW RISK] Safe. Can skip up to 17 classes, but regular attendance
is advised.
Bhavana       [MEDIUM RISK] Attend 27 of 35 remaining classes; avoid more than 8
absences.
Charan        [CRITICAL] Cannot reach 75% even with full attendance. Apply for
condonation / meet mentor.
Divya         [LOW RISK] Safe. Can skip up to 20 classes, but regular attendance
is advised.
Elango        [CRITICAL] Cannot reach 75% even with full attendance. Apply for
condonation / meet mentor.
Farah         [HIGH RISK] Attend at least 30 of 45 remaining classes.
Counselling recommended.

=====
=====
SUMMARY
=====
=====
LOW RISK      : 2
MEDIUM RISK  : 1
HIGH RISK     : 1
CRITICAL      : 2
Highest risk student: Charan (100.00%)
```